CSC_51054_EP - Data Challenge
Influencer or Observer: Predicting Social Roles

November 2025

1 Introduction

The goal of this data challenge is to explore and apply machine learning techniques to a real-world classification problem involving social media behavior. Specifically, this challenge focuses on the ability to predict a user's social role—either an **Influencer** or an **Observer**—based solely on the content of their tweets. Influencers are characterized by a high-leverage, asymmetrical network (a high follower-to-following ratio), while Observers maintain a more balanced, reciprocal network. Intuitively, Influencers are users who tend to accumulate a large audience and broadcast content widely, whereas Observers are users who engage in reciprocal conversations and build more balanced social interactions. This distinction creates a rich opportunity to study how language usage reflects underlying social strategies, and to design models capable of detecting these roles from text and general user features alone.

You are provided with a dataset in which each row corresponds to a single tweet, and each tweet is labeled with a binary value: `1` if it was authored by an Influencer, and `0` if it was authored by an Observer. While individual users may have multiple tweets, the challenge is structured to allow predictions based on a single tweet, emphasizing the ability of models to capture role-specific language patterns in isolation. This setup introduces challenges related to text preprocessing, feature extraction, and the selection of appropriate modeling strategies.

The task encourages participants to experiment with a variety of approaches, ranging from traditional deep learning models such as multilayer perceptrons and recurrent neural networks, to modern deep learning techniques including transformers and language models fine-tuned for classification. Effective solutions will need to carefully consider the trade-offs between model complexity, interpretability, and generalization performance, while handling the variability and sparsity inherent in social media text. Hyperparameter tuning, regularization strategies, and thoughtful feature engineering will all play a key role in developing robust models capable of distinguishing between Influencers and Observers.

The primary evaluation metric for this challenge is **Accuracy**, which measures the proportion of correctly predicted labels. To participate, you create a Kaggle account. The URL to register for the competition and gain access to all relevant materials is the following:

https://www.kaggle.com/t/4badd0aaded241898e83387eab33cc31

2 Dataset Description

As part of the challenge, you are given the following files:

- **train.jsonl** – The training set, containing *38,560* users, *154,914* tweets, and *194* features.
- **kaggle_test.jsonl** – The test set, containing *25,890* users and *103,380* tweets.
- **logistic_predictions.csv** and **dummy.csv** – Sample submission files in the correct format for the two baseline classifiers provided.
- **baseline.ipynb** – A Jupyter notebook containing two simple baseline models: a Logistic Regression classifier and a Dummy Classifier that always predicts the most frequent class in the training set. This notebook demonstrates how to read and process the data and can be used as a starting point for developing your own models.

3 Provided Source Code

You are provided with two baseline classifiers, written in Python, to help you get started with the challenge.

The first is the **Dummy Classifier**, which always predicts the most frequent label in the training set. This simple approach serves as a reference point for model performance and achieves an accuracy of approximately **53%** on the public Kaggle leaderboard.

The second is a **Logistic Regression classifier** that uses only the text of each tweet. This baseline performs basic text preprocessing and then transforms the tweets into numerical features using **TF-IDF** vectorization. The accuracy of this model is approximately **63%**.

Both baselines treat each tweet independently, ignoring any temporal order or contextual relationship to the user's other tweets or the broader social network. These models are intended to provide a starting point for experimentation and illustrate how simple methods can perform on the dataset.

4 Submission Process

Submission files should be in **.csv format**, and contain two columns respectively named "ID" and "Prediction". The "ID" column is used to identify each data sample and therefore needs to appear as-is in the results file. The "Prediction" column takes 0 or 1 values and is the output of your classifier. Note that two sample submission files are available for download **(logistic_predictions.csv and dummy_predictions.csv)**. You can use it to test that everything works fine.

The competition ends on **Thursday 11th December at 17:00**. This is the deadline for you to submit a compressed file containing your source code and final report, explaining your solutions and discussing the scores you have achieved. Until then, you can submit your solution to Kaggle and get a score at most 15 times per day. There must be one final submission per team.

5 Grading Scheme

Each team has to be composed of 2 or 3 students. No larger or smaller teams will be accepted. Group choices need to be submitted to us by **Friday 21nd November at 17:00** at the link below:

https://forms.gle/AfhkFdV1K4U1j6eG6

Grading will be out of 100 points in total. Each team should deliver:

A submission on the Kaggle competition webpage. (20 points) will be allocated based on raw performance only, provided that the results are reproducible. That is, using only your code, the data provided on the competition page and *any additional resources you are able to reference and demonstrate understanding of*, the jury should be able to train your final model and use it to generate the predictions you submitted for scoring.

A zipped folder on moodle (30 points) including:

1. A folder named ”code” containing all the scripts needed to reproduce your submission.
2. A README file with brief instructions on how to run your code and where it expects the original data files.
3. A report (.pdf file), of max 3 pages, excluding the cover page and references. In addition to your self-contained 3-page report, you can use up to 3 extra pages of the appendix (for extra explanations, algorithms, figures, tables, etc.). Please ensure that both your real name(s) and the name of your Kaggle team appear on the cover page.

The report should be a maximum of three pages and clearly document your approach. Emphasis will be placed on **research rigor, experimentation, and reasoning**, not just raw predictive performance. You should describe how you preprocessed and represented the data, including modern NLP techniques such as TF-IDF, contextual embeddings, or other features. Explain the motivation behind each feature and any design choices, citing relevant research where appropriate. Your modeling approach could explore a range of methods, from classical machine learning to neural networks, language models, and graph-based techniques. Clearly explain your hyperparameter tuning strategies, steps to prevent overfitting, and the rationale for selecting your final model.

An oral presentation of your project and the achieved results (50 points) Oral presentations will be scheduled **in the week of the 15th of December**. A schedule on which you can choose presentation slots will be released after the team submissions have been made. We ask you to prepare **10-minute (sharp) presentations**, which will then be followed by questions from the examiners. It is required that all group members are present and take a speaking role during the presentation.

Please note that the **inclusion of any external data sources that directly contain the labels will result in you getting 0 points** on the kaggle submission and being penalised on the report and presentation.

6 Useful Python Libraries

To tackle this challenge effectively, you may find the following Python libraries useful:

- **Scikit-learn** – A versatile library for classical machine learning. Provides tools for preprocessing, feature extraction, and a wide range of models such as logistic regression, decision trees, and ensemble methods.
- **PyTorch** – A modern deep learning framework that allows you to build and train neural networks, including transformer-based language models. Its ecosystem (e.g., PyTorch Lightning, Hugging Face Transformers) simplifies experimentation and scaling.
- **Hugging Face Transformers** – Pre-trained transformer models (BERT, RoBERTa, GPT) for feature extraction, text classification, and fine-tuning on your dataset. Great for capturing nuanced language patterns.
- **NLTK / SpaCy** – Libraries for text preprocessing, tokenization, lemmatization, and other standard NLP tasks. SpaCy is optimized for speed and large-scale processing.
- **Gensim / FastText** – Tools for unsupervised word embeddings and topic modeling. Useful if you want to generate embeddings without training large models from scratch.
- **SentenceTransformers** – Provides ready-to-use sentence and document embeddings for semantic analysis, clustering, and classification tasks.
- **NetworkX** – A library for creating and analyzing graphs. Useful if you want to explore user networks or interactions as a graph structure.
- **PyTorch Geometric (PyG) / DGL** – Advanced libraries for graph neural networks, ideal if you plan to incorporate social network structures into your model.